

```

        if newTimeInSeconds != None:
            self.timeInSeconds = newTimeInSeconds
        self.running = True
        self.startTime = time.time()

    ❸ def update(self):
        --- snip ---
        if not self.running:
            return False
        self.savedSecondsElapsed = time.time() - self.startTime
        if self.savedSecondsElapsed < self.timeInSeconds:
            return False # running but hasn't reached limit

        else: # timer has finished
            self.running = False
            if self.callBack is not None:
                self.callBack(self.nickname)

            return True # True here means that the timer has ended

    ❹ def getTime(self):
        --- snip ---
        if self.running:
            self.savedSecondsElapsed = time.time() - self.startTime

        return self.savedSecondsElapsed

    ❺ def stop(self):
        """Stops the timer"""
        self.getTime() # remembers final self.savedSecondsElapsed
        self.running = False

```

Listing 13-2: A simple `Timer` class

When you create a `Timer` object, the only required argument is the number of seconds you want the timer to run ❶. You can optionally supply a nickname for the timer and a function or method to be called back when the time has elapsed. If you specify a callback, the nickname will be passed in when the callback happens.

You call the `start()` method ❷ to start the timer running. The `Timer` object remembers the start time in the instance variable `self.startTime`.